PM Digest — Multi-Agent Orchestration

Slack + Gmail + Calendar → one synthesized daily digest, built as an n8n multi-agent workflow.

n8n | GPT-4o-mini | Slack API | Gmail API | Google Calendar API | Google Sheets

The Problem

Scattered across tools

Decisions, deadlines, and blockers live in Slack, Gmail, and Calendar separately — none of them talk to each other.

Cross-source links get missed

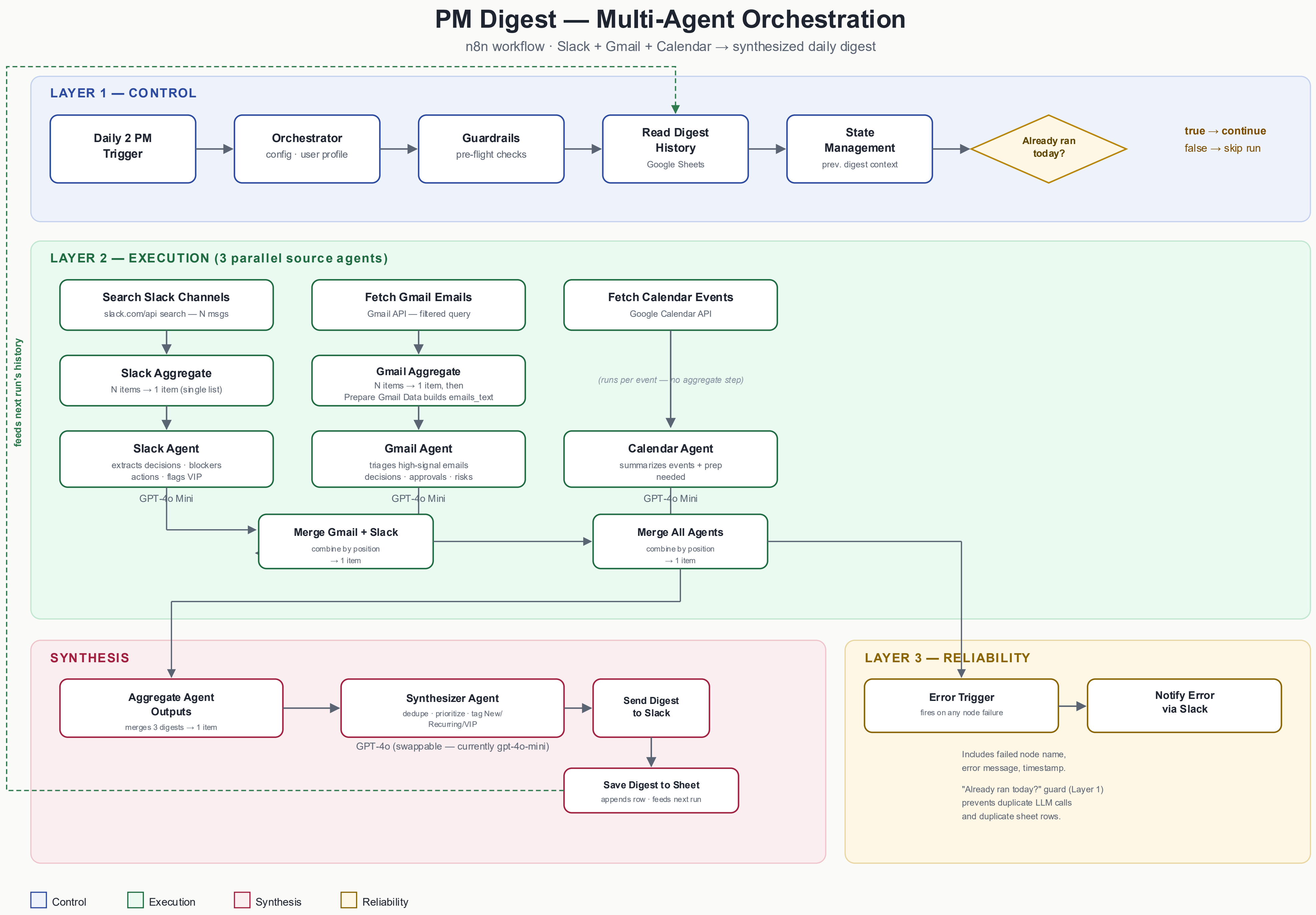
A deadline mentioned in an email is often the same thread being discussed in Slack — easy to miss when checked in isolation.

Single-agent chat isn't enough

A chatbot that answers one question at a time doesn't ingest multiple sources at once or reconcile them into one picture.

Objective: ingest 3 sources in parallel, route each through a dedicated agent, and synthesize one prioritized, deduplicated digest — automatically, on a schedule.

System Flow


PM Digest — Multi-Agent Orchestration
n8n workflow · Slack + Gmail + Calendar → synthesized daily digest
LAYER 1 — CONTROL
Daily 2 PM Trigger
Orchestrator
config · user profile
Guardrails
pre-flight checks
Read Digest History
Google Sheets
State Management
prev. digest context
Already ran today?
true → continue
false → skip run
feeds next run's history
LAYER 2 — EXECUTION (3 parallel source agents)
Search Slack Channels
slack.com/api search — N msgs
Fetch Gmail Emails
Gmail API — filtered query
Fetch Calendar Events
Google Calendar API
Slack Aggregate
N items → 1 item (single list)
Gmail Aggregate
N items → 1 item, then
Prepare Gmail Data builds emails_text
(runs per event — no aggregate step)
Slack Agent
extracts decisions · blockers
actions · flags VIP
Gmail Agent
triages high-signal emails
decisions · approvals · risks
Calendar Agent
summarizes events + prep
needed
GPT-4o Mini
GPT-4o Mini
GPT-4o Mini
Merge Gmail + Slack
combine by position
→ 1 item
Merge All Agents
combine by position
→ 1 item
SYNTHESIS
Aggregate Agent Outputs
merges 3 digests → 1 item
Synthesizer Agent
dedupe · prioritize · tag New/
Recurring/VIP
GPT-4o (swappable — currently gpt-4o-mini)
Send Digest to Slack
Save Digest to Sheet
appends row · feeds next run
LAYER 3 — RELIABILITY
Error Trigger
fires on any node failure
Notify Error via Slack
Includes failed node name,
error message, timestamp.
"Already ran today?" guard (Layer 1)
prevents duplicate LLM calls
and duplicate sheet rows.
Control
Execution
Synthesis
Reliability

Key Decisions & Trade-offs

1 Batch before you summarize

Multi-item sources (Slack, Gmail) run through an Aggregate node before the LLM call — one agent run per source, not one per message/email. Without this, item-count mismatches silently multiplied every downstream step.

2 Guard early, not late

The "already ran today?" check sits right after State Management, before any source agent fires — a same-day re-run costs zero API/LLM calls instead of being discarded at the last step.

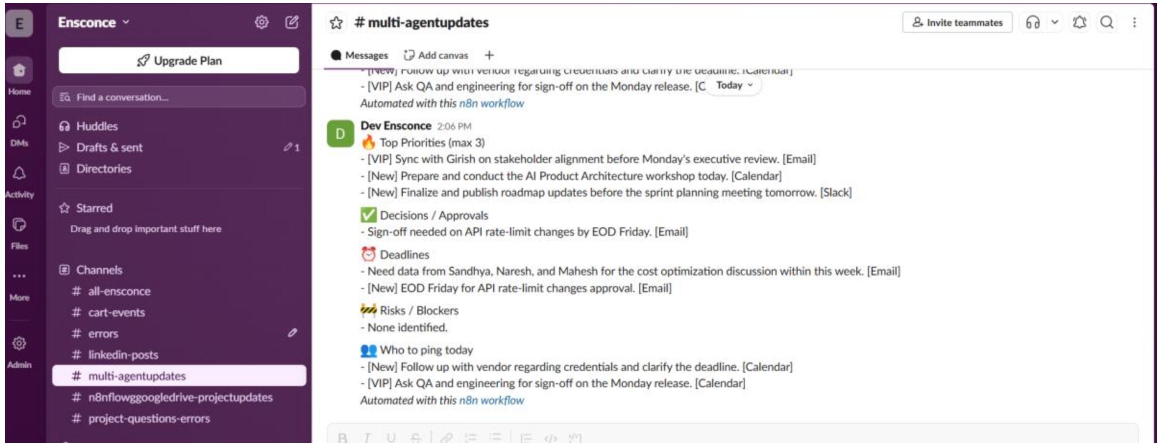
3 Match model to task difficulty

Source agents run gpt-4o-mini for fast, structured extraction. The Synthesizer is wired for gpt-4o — deduping and prioritizing across 3 sources is a harder reasoning task, and the model is swappable per cost/quality needs.

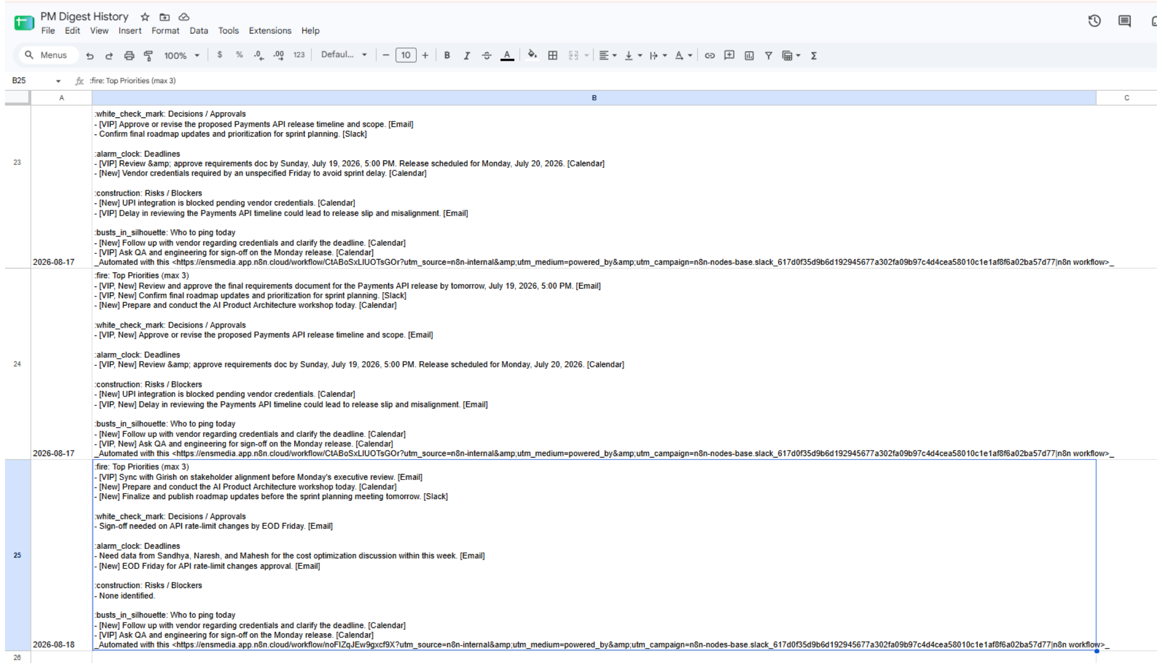
4 Fail loud, not silent

A dedicated Error Trigger branch posts the failed node name, error message, and timestamp to Slack — a broken source is visible immediately, not discovered the next morning.

Outcome



Digest posted to Slack



History logged to Google Sheets

What held up

The 3-layer split (Control / Execution / Reliability) made debugging tractable — every failure was isolable to config, a single agent, or the merge/synthesis step.

What I'd do next

Aggregate the Calendar branch too — it still runs its agent once per event. Fine at low volume, but the same fan-out risk resurfaces on a busier calendar day.